Pattern Recognition

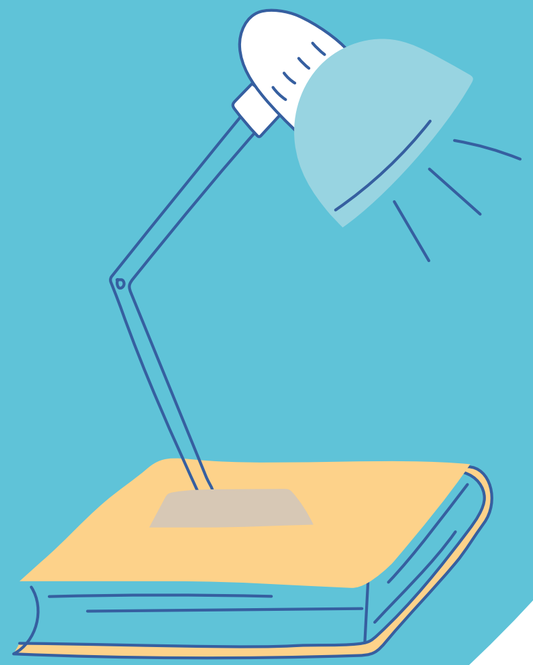
Hand Gesture Recognition

Group No. : 26

Team Members:

Rana Eissa
202101216

Ahmed Alaa
202100031



Contents

Project idea & problem definition:

Our project addresses the main problem of improving human-computer interaction through precise hand gesture recognition. The project's scope includes creating a Hand Gesture Database using near-infrared images captured by the Leap Motion sensor.

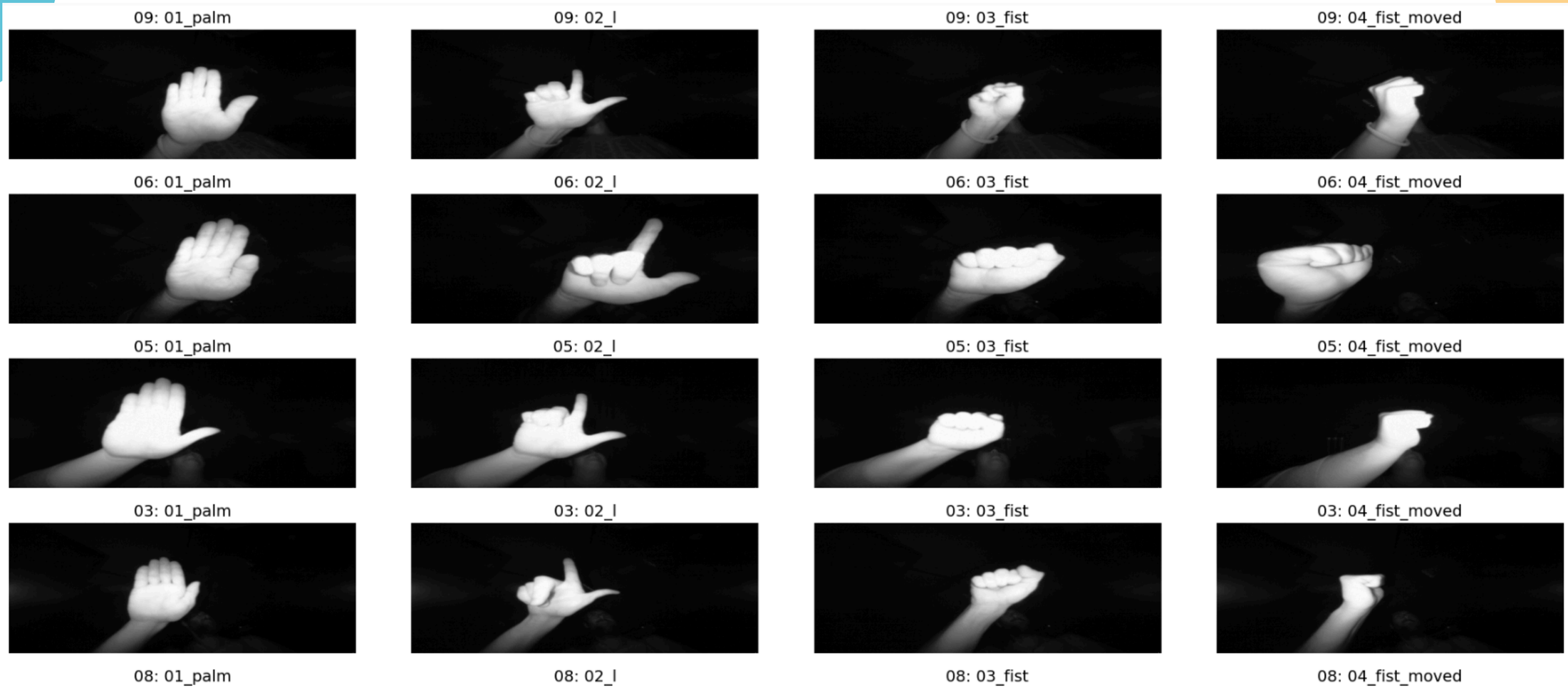
The main challenge that our project tackles is developing a machine learning model capable of accurately recognizing these hand gestures. By leveraging the dataset of near-infrared images, we aim to train a sophisticated recognition system that can interpret gestures and translate them into actionable commands or actions.

Our target solution involves employing advanced recognition methods within the machine learning model. This includes techniques such as deep learning algorithms to analyze the intricate patterns and nuances of hand gestures, ultimately enhancing user experience across various domains.

Through this project, we seek to revolutionize interaction modalities in virtual reality, gaming, healthcare, and other human-computer interaction systems by offering a reliable and precise hand gesture recognition system.

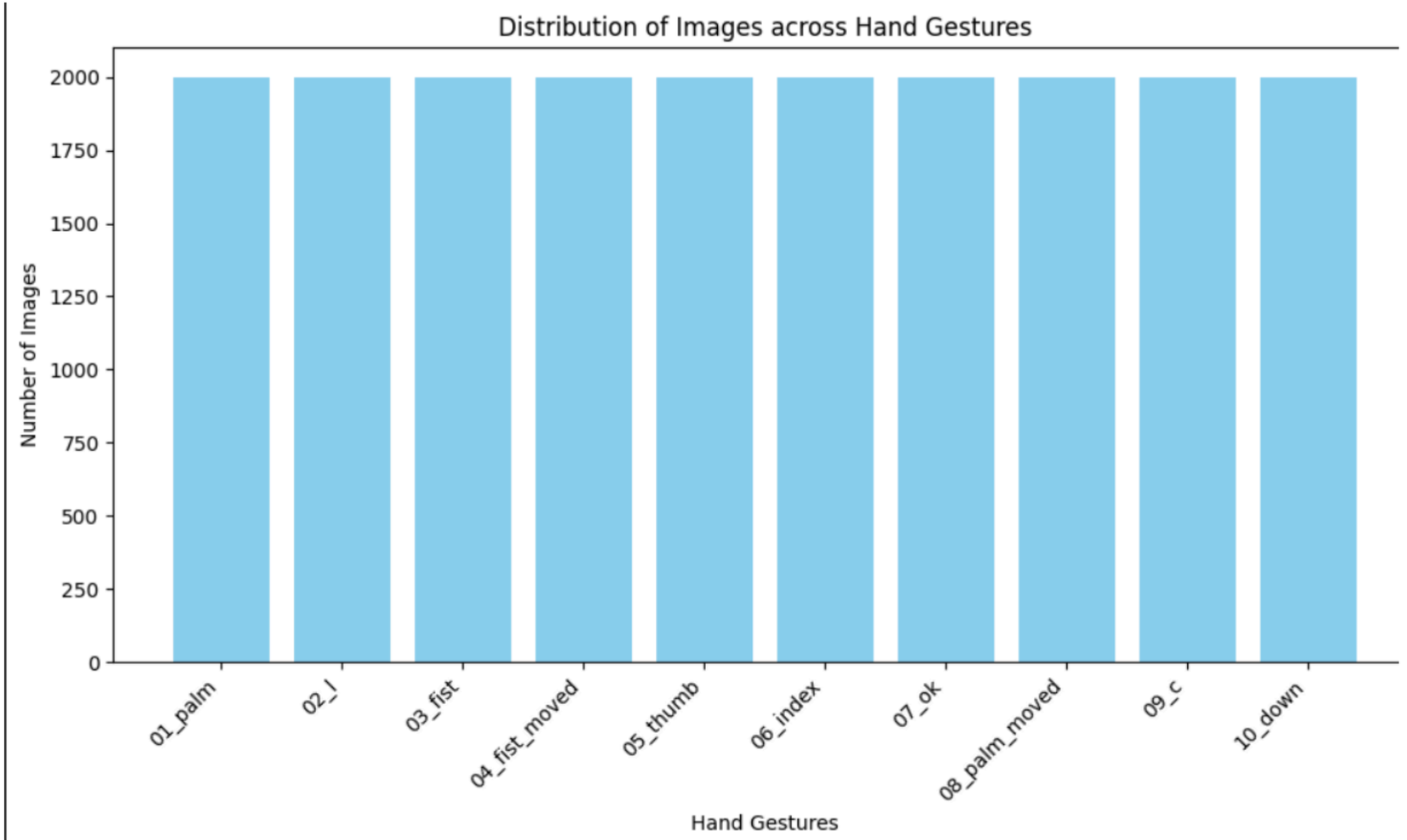
Dataset description

Our dataset type is images, available on Kaggle consisting of 10 distinct hand gestures, namely **'01_palm', '02_l', '03_fist', '04_fist_moved', '05_thumb', '06_index', '07_ok', '08_palm_moved', '09_c', and '10_down'**,
each identified by directories labeled from '00' to '09'. Each hand gesture class contains 200 images per subject, resulting in a total of 20,000 images in the dataset.



Challenges in the dataset: The hands details were a bit blurry, so we needed to do a preprocessing step we'll se later on.

Data exploration



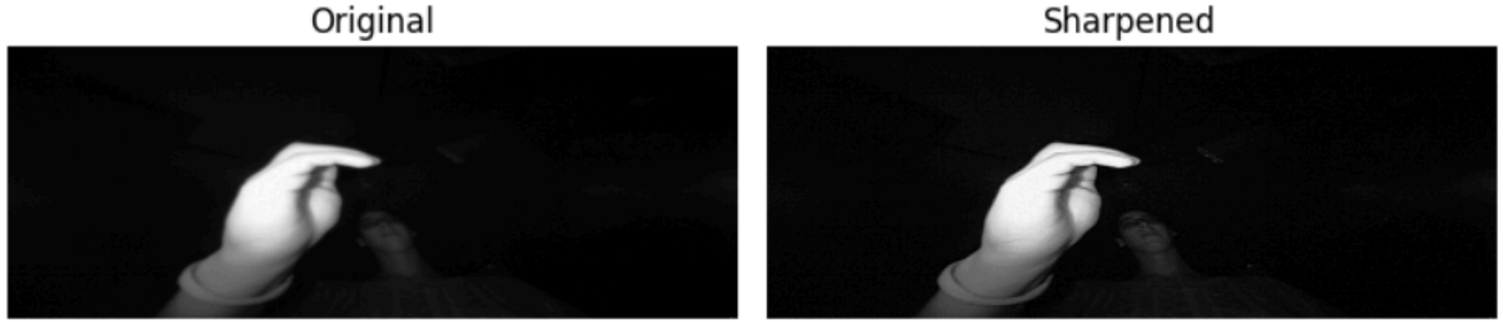

```
All images have the same size: (240, 640)
```

```
All images have the same extension: .png
```

Pre-processing & Features

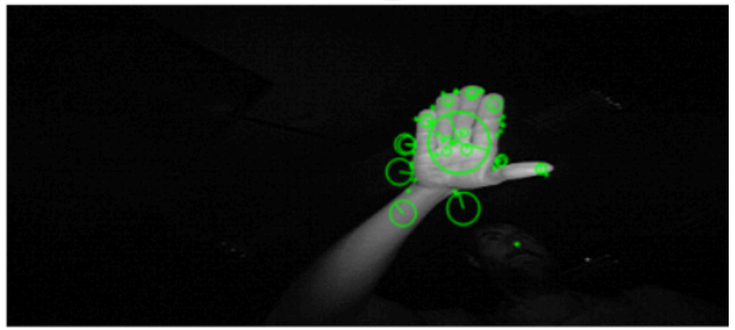
Sharpness

As previously mentioned, we encountered a minor challenge regarding the clarity of details within the hands, prompting us to apply sharpening techniques to enhance their visibility.



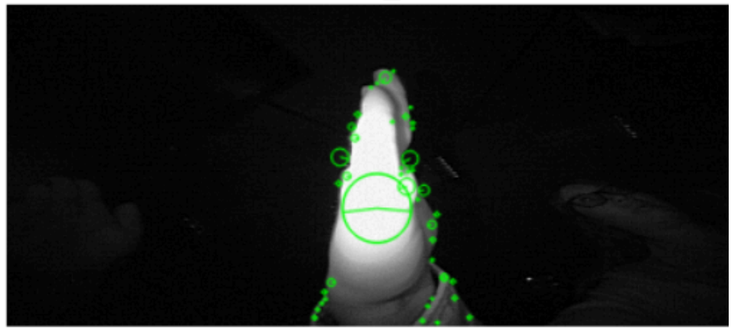
rpened

04: 02_l - Keypoints



04: 02_l - Sharpened

04: 02_l - Keypoints



SIFT

We used SIFT as its features have good repeatability and distinctiveness, and is prone to scale changes, rotations, and any background clutters making them effective for matching and identifying key points in hand gesture images, which is crucial for recognition tasks.

Total number of images processed: 20000
Total number of Descriptors: 1222820
Number of SIFT features extracted: 1132000

Shape of features array: (20000, 16384)

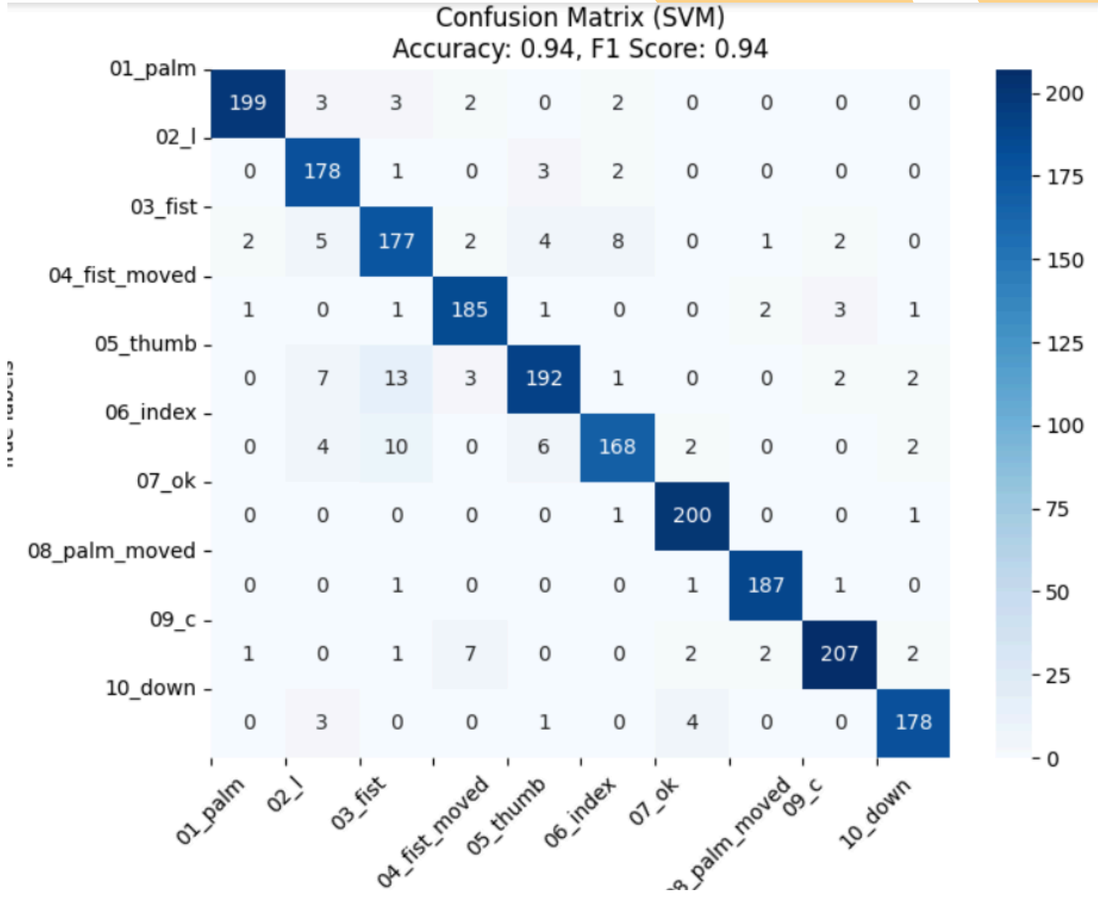
Machine Learning

1: SVM

In hand recognition, SVM works by creating an optimal hyperplane that maximizes the margin between different hand gesture classes. This hyperplane effectively separates hand gestures into distinct categories, allowing for clear decision boundaries. By identifying key features and patterns in hand gesture data, SVM facilitates accurate classification, enabling systems to recognize and differentiate between various hand gestures with high precision.

```
Accuracy on Validation Set (SVM): 0.94
Accuracy on Test Set (SVM): 0.94
```

Class Name	Accuracy	Sensitivity	Specificity	Precision	F1 Score
01_palm	0.993	0.952153	0.997767	0.980296	0.966019
02_l	0.986	0.967391	0.987885	0.89	0.927083
03_fist	0.973	0.880597	0.983324	0.855072	0.867647
04_fist_moved	0.9885	0.953608	0.992248	0.929648	0.941476
05_thumb	0.9785	0.872727	0.991573	0.927536	0.899297
06_index	0.981	0.875	0.992257	0.923077	0.898396
07_ok	0.9945	0.990099	0.994994	0.956938	0.973236
08_palm_moved	0.996	0.984211	0.997238	0.973958	0.979058
09_c	0.9885	0.932432	0.995501	0.962791	0.947368
10_down	0.992	0.956989	0.99559	0.956989	0.956989



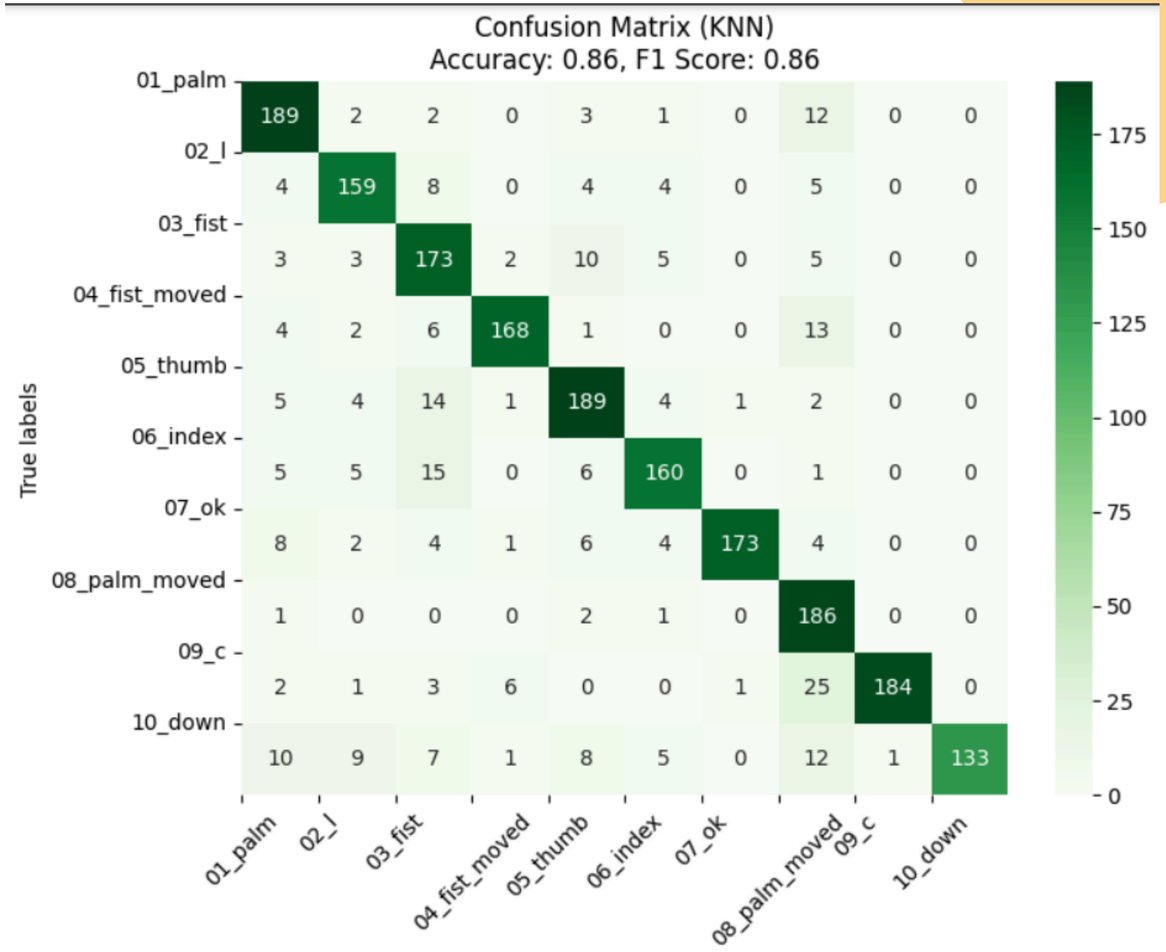
Machine learning

2: KNN

K-Nearest Neighbors (KNN) is used in hand gesture recognition to classify gestures based on their similarity to nearby data points. It works by identifying the k nearest neighbors (data points with similar features) to a given hand gesture image and assigning the majority class among these neighbors as the predicted class for the input image. KNN is effective for hand gesture recognition as it relies on the idea that similar gestures are likely to belong to the same class.

```
Accuracy on Validation Set: 0.86
Accuracy on Test Set: 0.86
```

Class Name	Accuracy	Sensitivity	Specificity	Precision	F1 Score
01_palm	0.969	0.904306	0.976549	0.818182	0.859091
02_l	0.9735	0.86413	0.984581	0.850267	0.857143
03_fist	0.9565	0.860697	0.967204	0.74569	0.799076
04_fist_moved	0.9815	0.865979	0.993909	0.938547	0.900804
05_thumb	0.9645	0.859091	0.977528	0.825328	0.841871
06_index	0.972	0.833333	0.986726	0.869565	0.851064
07_ok	0.9845	0.856436	0.998888	0.988571	0.917772
08_palm_moved	0.9585	0.978947	0.956354	0.701887	0.817582
09_c	0.9805	0.828829	0.999438	0.994595	0.904177
10_down	0.9735	0.715054	1	1	0.833856

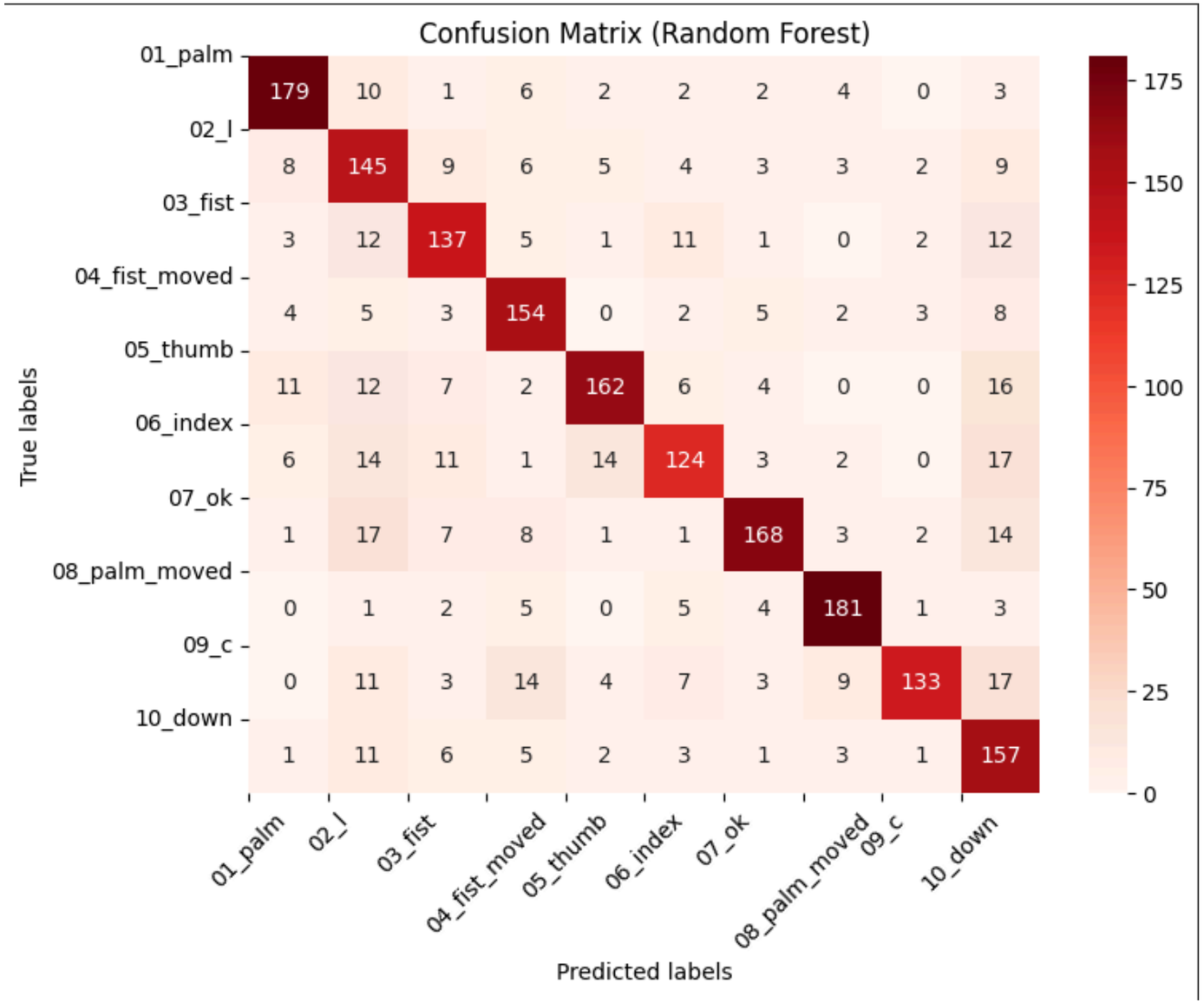


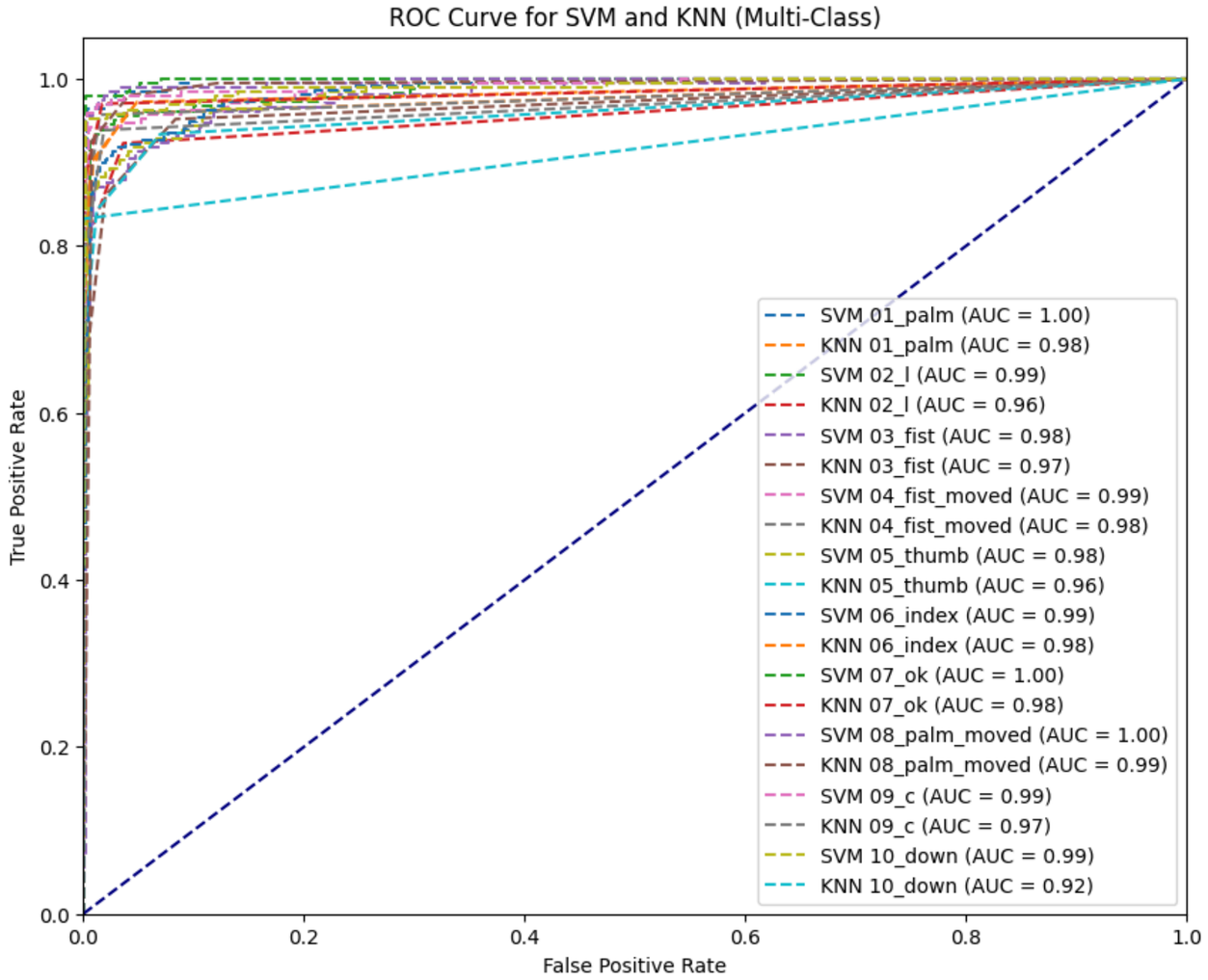
Machine learning

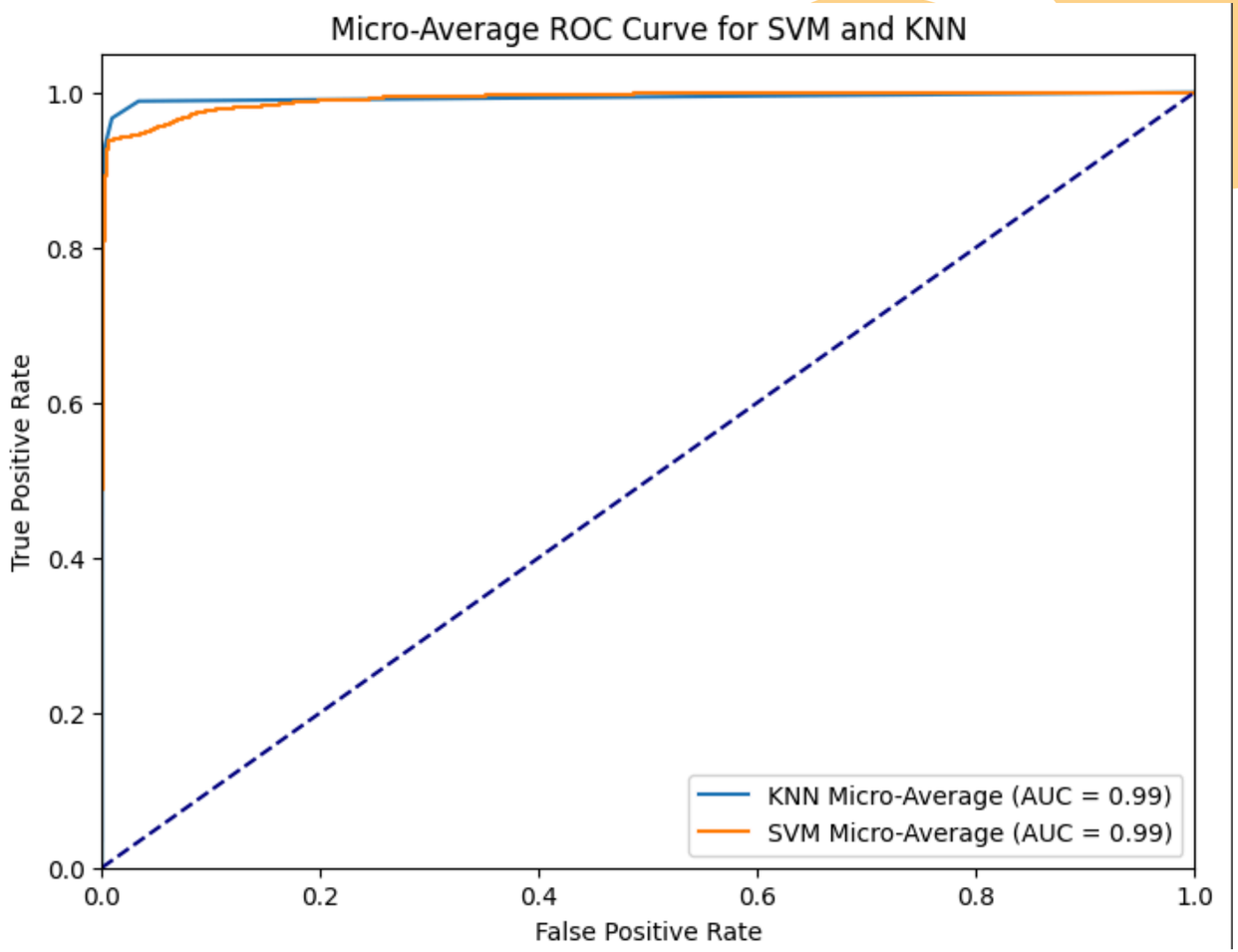
3: Random Forest

Random Forest is employed in hand gesture recognition for its ability to handle complex datasets with high-dimensional feature spaces effectively. It works by constructing multiple decision trees during training and then averaging their predictions to improve accuracy and reduce overfitting. It leverages the diversity of decision trees to capture different aspects and variations within the gestures.

```
Accuracy on Validation Set (Random Forest): 0.77
Accuracy on Test Set (Random Forest): 0.75
```


Confusion Matrix (Random Forest)
True labels
Predicted labels
01_palm
02_l
03_fist
04_fist_moved
05_thumb
06_index
07_ok
08_palm_moved
09_c
10_down
179 10 1 6 2 2 2 4 0 3
8 145 9 6 5 4 3 3 2 9
3 12 137 5 1 11 1 0 2 12
4 5 3 154 0 2 5 2 3 8
11 12 7 2 162 6 4 0 0 16
6 14 11 1 14 124 3 2 0 17
1 17 7 8 1 1 168 3 2 14
0 1 2 5 0 5 4 181 1 3
0 11 3 14 4 7 3 9 133 17
1 11 6 5 2 3 1 3 1 157
175
150
125
100
75
50
25
0


ROC Curve for SVM and KNN (Multi-Class)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
SVM 01_palm (AUC = 1.00)
KNN 01_palm (AUC = 0.98)
SVM 02_l (AUC = 0.99)
KNN 02_l (AUC = 0.96)
SVM 03_fist (AUC = 0.98)
KNN 03_fist (AUC = 0.97)
SVM 04_fist_moved (AUC = 0.99)
KNN 04_fist_moved (AUC = 0.98)
SVM 05_thumb (AUC = 0.98)
KNN 05_thumb (AUC = 0.96)
SVM 06_index (AUC = 0.99)
KNN 06_index (AUC = 0.98)
SVM 07_ok (AUC = 1.00)
KNN 07_ok (AUC = 0.98)
SVM 08_palm_moved (AUC = 1.00)
KNN 08_palm_moved (AUC = 0.99)
SVM 09_c (AUC = 0.99)
KNN 09_c (AUC = 0.97)
SVM 10_down (AUC = 0.99)
KNN 10_down (AUC = 0.92)


Micro-Average ROC Curve for SVM and KNN
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
KNN Micro-Average (AUC = 0.99)
SVM Micro-Average (AUC = 0.99)

Deep learning

1: CNN

Convolutional Neural Networks (CNNs) are extensively utilized in hand gesture recognition due to their ability to automatically learn hierarchical features from raw pixel data. CNNs excel at capturing spatial patterns and relationships within images, making them well-suited for tasks like gesture recognition where local features and their arrangements are crucial for classification. By leveraging 5 convolutional layers, 5 pooling layers, and activations of type Relu, CNNs can extract meaningful representations of hand gestures, enabling accurate and efficient recognition performance.

total = 19
5 Convolutional Layers
5 MaxPooling2D Layers
5 Dropout Layers
1 Flatten Layer
3 Dense Layers

Input Layer:

Shape: (150, 150, 3)

Purpose: Receives RGB images with a height and width of 150 pixels and 3 color channels (red, green, blue).

Convolutional Layers:

Conv2D:

Filters: Extracts features using convolutional operations with 3x3 filter kernels.

Activation: ReLU (Rectified Linear Unit) activation function applied to introduce non-linearity.

Padding: 'Same' padding is used to preserve spatial dimensions.

MaxPooling2D:

Pool Size: Performs max pooling with a 2x2 window to downsample feature maps.

Stride: Moves the pooling window by 2 pixels horizontally and vertically.

Dropout:

Rate: Drops 30% of neurons during training to prevent overfitting.

Flatten Layer:

Purpose: Converts the output from convolutional layers into a 1D array for input to fully connected layers.

Fully Connected Layers:

Dense:

Neurons: 1024 neurons with ReLU activation function to learn complex patterns.

Dropout: Regularizes the network by dropping 50% of neurons during training.

Dense:

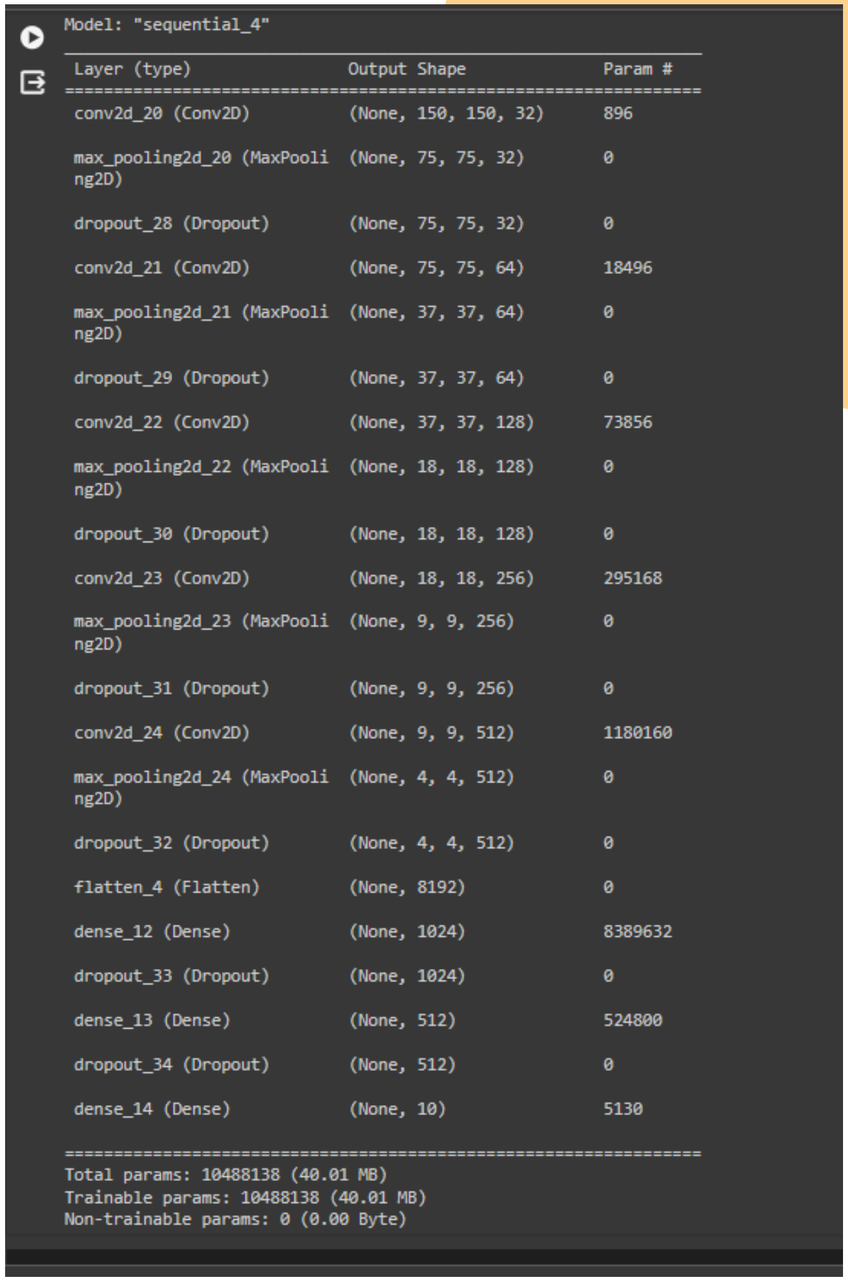
Neurons: 512 neurons with ReLU activation function for further feature learning.

Dropout: Adds regularization by dropping 50% of neurons.

Dense (Output Layer):

Neurons: 10 neurons corresponding to the number of classes in the classification task.

Activation: Softmax activation function to compute class probabilities.

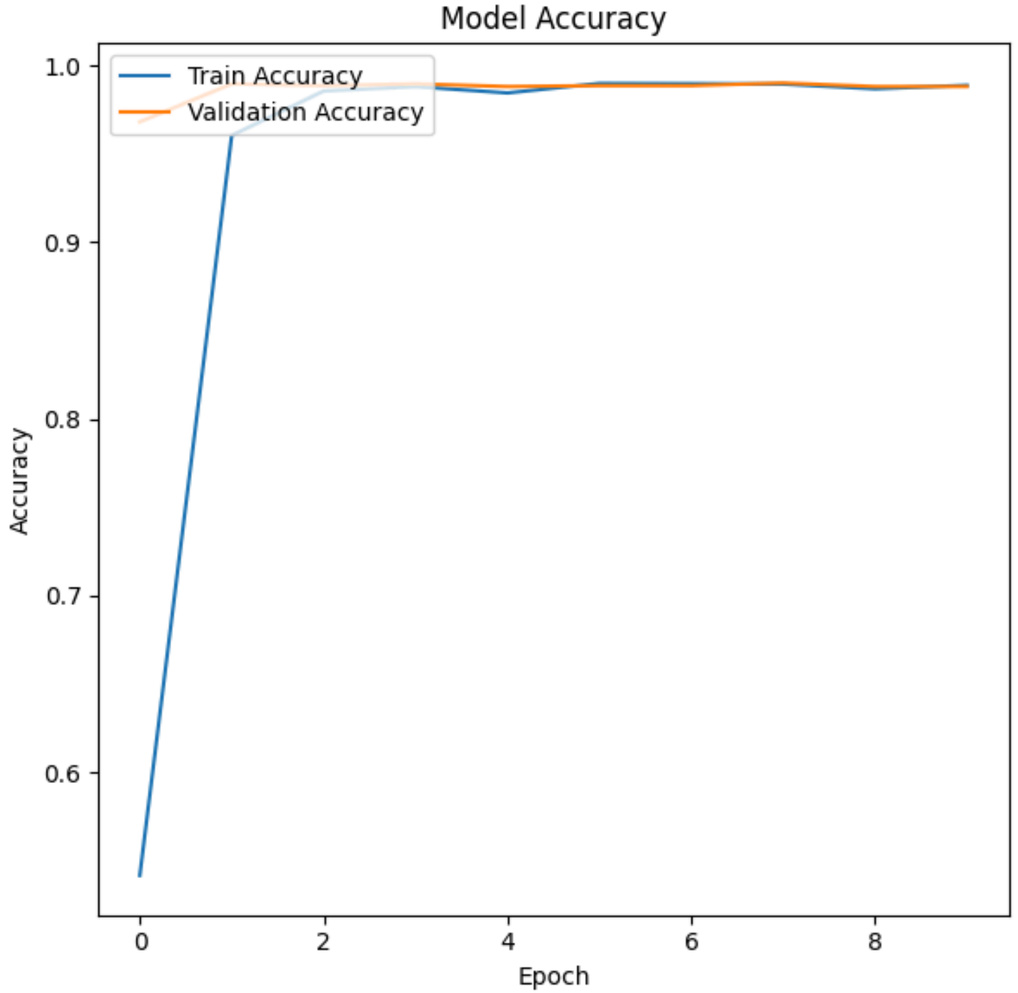


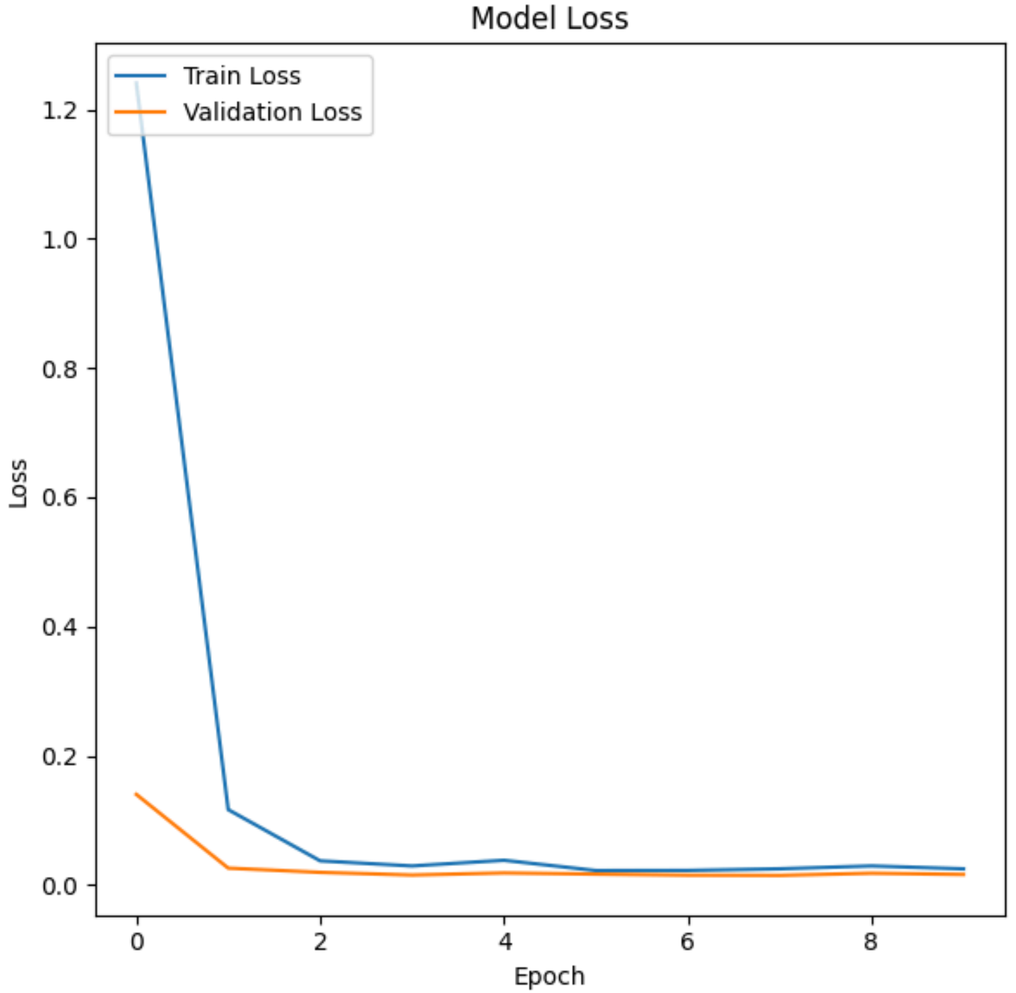
```
Model: "sequential_4"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_20 (Conv2D)          (None, 150, 150, 32)      896

 max_pooling2d_20 (MaxPooli  (None, 75, 75, 32)        0
 ng2D)

 dropout_28 (Dropout)        (None, 75, 75, 32)        0

 conv2d_21 (Conv2D)          (None, 75, 75, 64)        18496

 max_pooling2d_21 (MaxPooli  (None, 37, 37, 64)        0
 ng2D)

 dropout_29 (Dropout)        (None, 37, 37, 64)        0

 conv2d_22 (Conv2D)          (None, 37, 37, 128)       73856

 max_pooling2d_22 (MaxPooli  (None, 18, 18, 128)       0
 ng2D)

 dropout_30 (Dropout)        (None, 18, 18, 128)       0

 conv2d_23 (Conv2D)          (None, 18, 18, 256)       295168

 max_pooling2d_23 (MaxPooli  (None, 9, 9, 256)         0
 ng2D)

 dropout_31 (Dropout)        (None, 9, 9, 256)         0

 conv2d_24 (Conv2D)          (None, 9, 9, 512)         1180160

 max_pooling2d_24 (MaxPooli  (None, 4, 4, 512)         0
 ng2D)

 dropout_32 (Dropout)        (None, 4, 4, 512)         0

 flatten_4 (Flatten)         (None, 8192)              0

 dense_12 (Dense)            (None, 1024)              8389632

 dropout_33 (Dropout)        (None, 1024)              0

 dense_13 (Dense)            (None, 512)               524800

 dropout_34 (Dropout)        (None, 512)               0

 dense_14 (Dense)            (None, 10)                5130

=================================================================
Total params: 10488138 (40.01 MB)
Trainable params: 10488138 (40.01 MB)
Non-trainable params: 0 (0.00 Byte)
```

```
Epoch 1/10
250/250 [==============================] - 48s 163ms/step - loss: 1.1915 - accuracy: 0.5571 - precision: 0.8423 - recall: 0.4306 - val_loss: 0.2289 - val_accuracy: 0.9370
Epoch 2/10
250/250 [==============================] - 39s 156ms/step - loss: 0.0865 - accuracy: 0.9697 - precision: 0.9731 - recall: 0.9659 - val_loss: 0.0202 - val_accuracy: 0.9890
Epoch 3/10
250/250 [==============================] - 37s 148ms/step - loss: 0.0343 - accuracy: 0.9859 - precision: 0.9865 - recall: 0.9853 - val_loss: 0.0336 - val_accuracy: 0.9840
Epoch 4/10
250/250 [==============================] - 40s 158ms/step - loss: 0.0289 - accuracy: 0.9881 - precision: 0.9884 - recall: 0.9876 - val_loss: 0.0159 - val_accuracy: 0.9890
Epoch 5/10
250/250 [==============================] - 40s 160ms/step - loss: 0.0177 - accuracy: 0.9927 - precision: 0.9929 - recall: 0.9922 - val_loss: 0.0147 - val_accuracy: 0.9905
Epoch 6/10
250/250 [==============================] - 37s 148ms/step - loss: 0.0300 - accuracy: 0.9883 - precision: 0.9886 - recall: 0.9874 - val_loss: 0.0158 - val_accuracy: 0.9885
Epoch 7/10
250/250 [==============================] - 40s 160ms/step - loss: 0.0198 - accuracy: 0.9907 - precision: 0.9910 - recall: 0.9904 - val_loss: 0.0154 - val_accuracy: 0.9905
Epoch 8/10
250/250 [==============================] - 37s 150ms/step - loss: 0.0177 - accuracy: 0.9902 - precision: 0.9904 - recall: 0.9899 - val_loss: 0.0204 - val_accuracy: 0.9885
Epoch 9/10
250/250 [==============================] - 40s 159ms/step - loss: 0.0243 - accuracy: 0.9889 - precision: 0.9892 - recall: 0.9884 - val_loss: 0.0180 - val_accuracy: 0.9915
Epoch 10/10
250/250 [==============================] - ETA: 0s - loss: 0.0371 - accuracy: 0.9864 - precision: 0.9872 - recall: 0.9859
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.00010000000474974513.
```

```
Found 2000 images belonging to 10 classes.
31/31 [==============================] - 4s 137ms/step - loss: 0.0165 - accuracy: 0.9879 - precision: 0.9879 - recall: 0.9879
Validaion Loss: 0.01653689704835415
Validaion Accuracy: 0.9879032373428345
Validaion Precision: 0.9879032373428345
Validaion Recall: 0.9879032373428345
```

```
Found 2000 images belonging to 10 classes.
31/31 [==============================] - 3s 110ms/step - loss: 0.0128 - accuracy: 0.9929 - precision: 0.9929 - recall: 0.9929
Test Loss: 0.012835155241191387
Test Accuracy: 0.992943525314331
Test Precision: 0.992943525314331
Test Recall: 0.992943525314331
```


Model Accuracy
Train Accuracy
Validation Accuracy
Accuracy
Epoch
1.0
0.9
0.8
0.7
0.6
0
2
4
6
8


Model Loss
Train Loss
Validation Loss
Loss
Epoch
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0
2
4
6
8

2: VGG16

VGG16, short for Visual Geometry Group 16, is a deep convolutional neural network architecture that has been widely used in various computer vision tasks, including hand gesture recognition. Its deep architecture consisting of multiple convolutional and pooling layers enables VGG16 to learn intricate hierarchical features from input images, capturing both low-level and high-level representations effectively. This makes VGG16 a suitable choice for hand gesture recognition tasks where extracting complex visual patterns and structures is essential for accurate classification.

total = 16
13 Convolutional Layers
5 MaxPooling2D Layers
5 Dropout Layers
1 Flatten Layer
3 Dense Layers

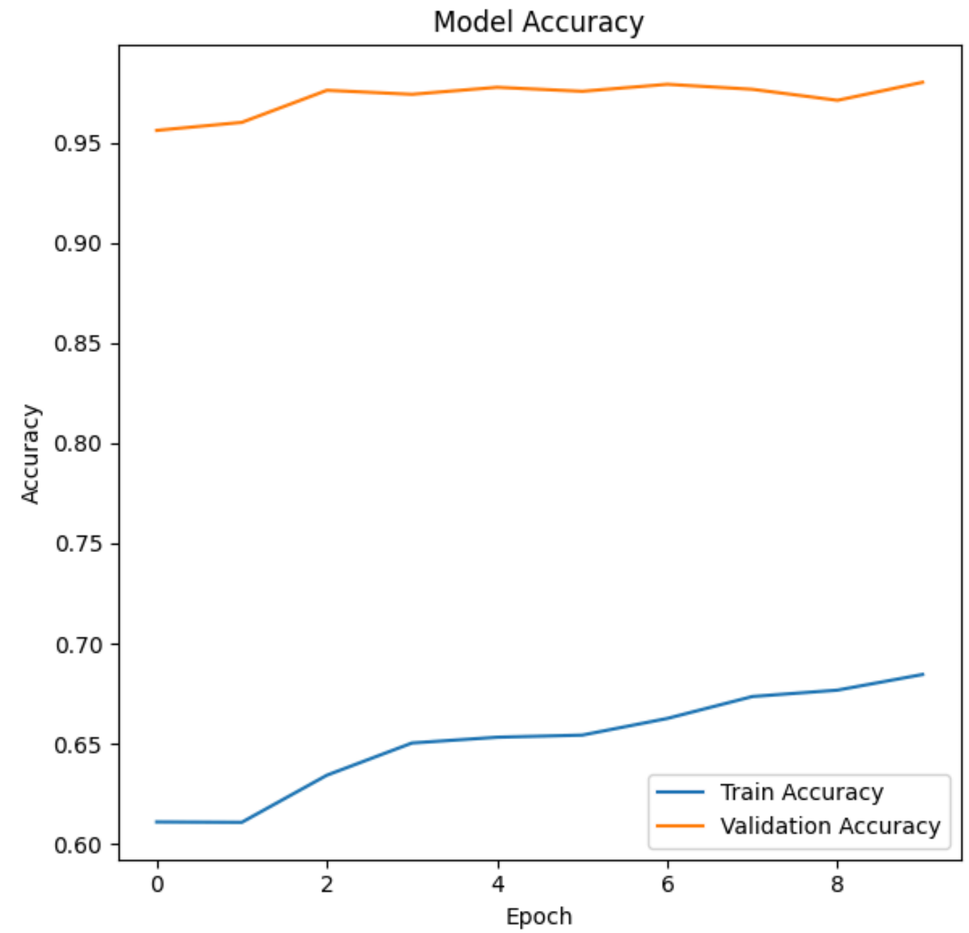
1. Input Layer:
   - Input Shape: (None, 224, 224, 3)
   - Description: Receives images with a height and width of 224 pixels and 3 color channels (RGB).
2. VGG16 Backbone:
   - Block 1:
     - Convolutional Layer: 2 layers with 64 filters each, using 3x3 filter size and ReLU activation.
     - MaxPooling2D Layer: Reduces spatial dimensions by half (112x112).
   - Block 2:
     - Convolutional Layer: 2 layers with 128 filters each, using 3x3 filter size and ReLU activation.
     - MaxPooling2D Layer: Reduces spatial dimensions by half again (56x56).
   - Block 3:
     - Convolutional Layer: 3 layers with 256 filters each, using 3x3 filter size and ReLU activation.
     - MaxPooling2D Layer: Reduces spatial dimensions by half (28x28).
   - Block 4:
     - Convolutional Layer: 3 layers with 512 filters each, using 3x3 filter size and ReLU activation.
     - MaxPooling2D Layer: Reduces spatial dimensions by half (14x14).
   - Block 5:
     - Convolutional Layer: 3 layers with 512 filters each, using 3x3 filter size and ReLU activation.
     - MaxPooling2D Layer: Reduces spatial dimensions by half (7x7).
3. Additional Layers:
   - Flatten Layer: Flattens the output from the VGG16 backbone into a 1D array.
   - Dense Layer 1: Fully connected layer with 512 neurons and ReLU activation.
   - Dropout Layer: Drops 50% of neurons during training to reduce overfitting.
   - Dense Layer 2 (Output Layer): Dense layer with 10 neurons (corresponding to the number of classes) and softmax activation for classification.

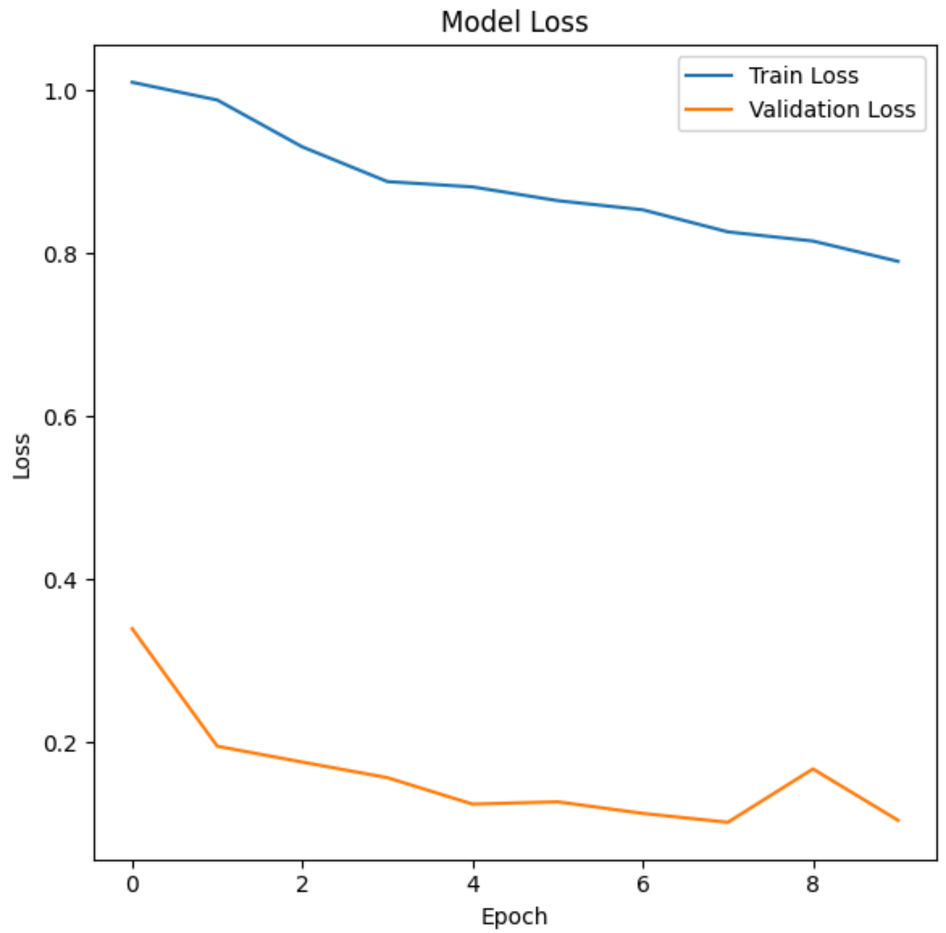
```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 224, 224, 3)]     0

 block1_conv1 (Conv2D)       (None, 224, 224, 64)      1792

 block1_conv2 (Conv2D)       (None, 224, 224, 64)      36928

 block1_pool (MaxPooling2D)  (None, 112, 112, 64)      0

 block2_conv1 (Conv2D)       (None, 112, 112, 128)     73856

 block2_conv2 (Conv2D)       (None, 112, 112, 128)     147584

 block2_pool (MaxPooling2D)  (None, 56, 56, 128)       0

 block3_conv1 (Conv2D)       (None, 56, 56, 256)       295168

 block3_conv2 (Conv2D)       (None, 56, 56, 256)       590080

 block3_conv3 (Conv2D)       (None, 56, 56, 256)       590080

 block3_pool (MaxPooling2D)  (None, 28, 28, 256)       0

 block4_conv1 (Conv2D)       (None, 28, 28, 512)       1180160

 block4_conv2 (Conv2D)       (None, 28, 28, 512)       2359808

 block4_conv3 (Conv2D)       (None, 28, 28, 512)       2359808

 block4_pool (MaxPooling2D)  (None, 14, 14, 512)       0

 block5_conv1 (Conv2D)       (None, 14, 14, 512)       2359808

 block5_conv2 (Conv2D)       (None, 14, 14, 512)       2359808

 block5_conv3 (Conv2D)       (None, 14, 14, 512)       2359808

 block5_pool (MaxPooling2D)  (None, 7, 7, 512)         0

 flatten (Flatten)           (None, 25088)             0

 dense (Dense)               (None, 512)               12845568

 dropout (Dropout)           (None, 512)               0

 dense_1 (Dense)             (None, 10)                5130

=================================================================
Total params: 27565386 (105.15 MB)
Trainable params: 12850698 (49.02 MB)
Non-trainable params: 14714688 (56.13 MB)
_________________________________________________________________
```

```
Epoch 1/10
250/250 [==============================] - 246s 982ms/step - loss: 1.0097 - accuracy: 0.6110 - val_loss: 0.3393 - val_accuracy: 0.9560 - lr: 0.0010
Epoch 2/10
250/250 [==============================] - 245s 979ms/step - loss: 0.9879 - accuracy: 0.6108 - val_loss: 0.1952 - val_accuracy: 0.9600 - lr: 0.0010
Epoch 3/10
250/250 [==============================] - 246s 982ms/step - loss: 0.9304 - accuracy: 0.6343 - val_loss: 0.1757 - val_accuracy: 0.9760 - lr: 0.0010
Epoch 4/10
250/250 [==============================] - 246s 985ms/step - loss: 0.8879 - accuracy: 0.6504 - val_loss: 0.1566 - val_accuracy: 0.9740 - lr: 0.0010
Epoch 5/10
250/250 [==============================] - 244s 976ms/step - loss: 0.8814 - accuracy: 0.6532 - val_loss: 0.1241 - val_accuracy: 0.9775 - lr: 0.0010
Epoch 6/10
250/250 [==============================] - 245s 982ms/step - loss: 0.8645 - accuracy: 0.6543 - val_loss: 0.1269 - val_accuracy: 0.9755 - lr: 0.0010
Epoch 7/10
250/250 [==============================] - 247s 985ms/step - loss: 0.8533 - accuracy: 0.6626 - val_loss: 0.1127 - val_accuracy: 0.9790 - lr: 0.0010
Epoch 8/10
250/250 [==============================] - 250s 1s/step - loss: 0.8262 - accuracy: 0.6736 - val_loss: 0.1018 - val_accuracy: 0.9765 - lr: 0.0010
Epoch 9/10
250/250 [==============================] - 242s 969ms/step - loss: 0.8149 - accuracy: 0.6768 - val_loss: 0.1673 - val_accuracy: 0.9710 - lr: 0.0010
Epoch 10/10
250/250 [==============================] - 245s 981ms/step - loss: 0.7900 - accuracy: 0.6846 - val_loss: 0.1042 - val_accuracy: 0.9800 - lr: 0.0010
```

```
32/32 [==============================] - 9s 265ms/step - loss: 0.0895 - accuracy: 0.9855
Test Loss: 0.0895
Test Accuracy: 0.9855
```

```
32/32 [==============================] - 8s 252ms/step - loss: 0.1042 - accuracy: 0.9800
Validation Loss: 0.1042
Validation Accuracy: 0.9800
```


Model Accuracy
Accuracy
Epoch
Train Accuracy
Validation Accuracy
0.60
0.65
0.70
0.75
0.80
0.85
0.90
0.95
0
2
4
6
8


Model Loss
Loss
Epoch
Train Loss
Validation Loss
0.2
0.4
0.6
0.8
1.0
0
2
4
6
8

Faced Challenges

We encountered difficulties while navigating through the directories and subfolders within the dataset, along with challenges in implementing one-vs-rest for our ROC analysis, leading us to opt for micro-average. One of the most demanding obstacles arose when facing limitations with GPU and RAM resources.

Thank you